

PIPE-RDF: Execution-Grounded Generation of Schema-Specific NL-SPARQL Benchmarks

Anonymous ACL submission

Abstract

Natural-language interfaces to Resource Description Framework (RDF) knowledge graphs require benchmarks whose SPARQL queries match the target schema, namespace conventions, and query distribution. Public knowledge-graph question answering (KGQA) datasets are useful for model comparison, but they are not designed to create evaluation sets for a new RDF schema. We present PIPE-RDF, a three-phase pipeline for constructing schema-specific natural-language-SPARQL (NL-SPARQL) benchmarks using reverse querying, category-balanced template generation, retrieval-augmented prompting, deduplication, and execution-based validation. We evaluate PIPE-RDF on two RDF schemas: a controlled company-location slice and a broader 25M-triple LDDB Semantic Publishing Benchmark (SPB) graph. The primary Qwen3.5-4B runs generate 3,600 Phase-3 pairs, exactly 200 per category for each schema across nine categories, with 100% parse validity, 100% execution validity, zero empty answers, and zero repairs under strict acceptance. Six cross-model probes with Qwen3.5-2B, 4B, and 9B produce balanced 450-pair benchmarks on both schemas with the same strict validity profile. A downstream utility study shows that category-retrieval prompting with generated examples substantially improves NL-to-SPARQL execution and answer accuracy over schema-only zero-shot prompting. Code is available at <https://anonymous.4open.science/status/PIPE-RDF-B750>.

1 Introduction

RDF knowledge graphs expose structured data through SPARQL, but natural-language access to those graphs remains highly schema-dependent. A natural-language question can be fluent while its corresponding query uses the wrong predicate, assumes a missing type, or returns an answer set that cannot exist in the target graph. This makes

benchmark construction a first-order problem for NL-SPARQL work: an evaluation set must reflect the target ontology, prefixes, available attributes, and intended query categories before it can be used to compare parsers or retrieval strategies.

Public KGQA benchmarks such as LC-QuAD, QALD-9-plus, and Mintaka provide valuable shared test beds, but they are tied to fixed public schemas and entity distributions (Trivedi et al., 2017; Perevalov et al., 2022; Sen et al., 2022). They do not directly answer a common methodological question: given a new RDF schema, how can we construct a balanced, executable, and auditable NL-SPARQL benchmark without manually authoring every query? Modern large language models (LLMs) can help generate candidate questions and queries (Brown et al., 2020; Touvron et al., 2023), but unconstrained generation is prone to unsupported entities, invalid predicates, and semantically plausible but unanswerable queries.

PIPE-RDF addresses this benchmark-construction problem with a pipeline that grounds generation in graph facts. The central mechanism is reverse querying: before a question-query pair is accepted, the system first finds entity bindings that satisfy the intended template in the target graph. Category-balanced generation, retrieval-augmented prompting, predicate/type whitelists, deduplication, execution validation, and repair are then used to produce benchmark artifacts that are not merely grammatical, but executable against a fixed RDF endpoint.

Contributions. We:

- propose a three-phase pipeline for schema-specific NL-SPARQL benchmark generation with reverse-query grounding and execution validation;
- evaluate PIPE-RDF on a fixed company-location schema and a broader LDDB Semantic Publishing Benchmark (SPB) graph,

producing 3,600 strictly valid Phase-3 pairs across nine categories;

- report operational metrics including parse validity, execution success, empty-result rates, repair rates, latency, strategy coverage, and entity diversity;
- add cross-model robustness probes and a compact downstream utility evaluation for zero-shot versus retrieval-augmented NL-to-SPARQL prompting;
- provide a stratified semantic-audit packet and agreement tooling for independent human validation.

2 Background and Related Work

Benchmark coverage and query taxonomies. KGQA benchmarks often over-represent simple factoid queries. Mintaka explicitly balances query categories (counting, comparative, superlative, ordinal, multi-hop, intersection, difference/negation, yes/no, and generic) to avoid skewed evaluation (Sen et al., 2022). Public datasets such as LC-QuAD and QALD-9-plus provide valuable coverage but remain tied to public graphs and fixed schemas (Trivedi et al., 2017; Perevalov et al., 2022). Dynamic benchmarking efforts such as Dynabench further highlight the need for iterative, real-world evaluation (Kiela et al., 2021).

Retrieval augmentation and prompting. Retrieval-augmented generation (RAG) improves text-to-query performance by conditioning on relevant examples and schema cues (Lewis et al., 2021). In practice, RAG offers a low-cost alternative to fine-tuning when schema-specific examples are scarce. PIPE-RDF maintains category-specific retrieval banks to provide structurally similar NL-SPARQL pairs during generation, reducing cross-category drift and improving template grounding.

Validity and evaluation. Grammar-constrained decoding (e.g., PICARD) improves syntactic correctness for structured query generation (Scholak et al., 2021). SPARQL-specific metrics such as SP-BLEU/SP-F1 normalize variables for fairer matching (Rony et al., 2022), while code-oriented structural metrics such as CodeBLEU inspire future extensions (Ren et al., 2020). PIPE-RDF focuses on execution, parsing, and structural complexity metrics that are directly actionable for deployment and troubleshooting.

Feature	LC-QuAD	QALD	Mintaka	PIPE-RDF
Schema fidelity	Public	Public	Public	Fixed
Reverse querying	–	–	–	✓
Category balance	Partial	–	✓	✓
Repair loop	–	–	–	✓
Operational metrics	–	–	–	✓
RAG retrieval	–	–	–	✓

Table 1: Comparison of PIPE-RDF with existing KGQA benchmark approaches. PIPE-RDF combines reverse querying, category-aware RAG, and execution-based repair for schema-specific benchmark construction.

LLM-driven workload generation for analytics. Recent work on cross-model analytics workload generation uses a two-stage design (reverse query generation followed by NL query generation), explicit schema/strategy controls, and detailed error analyses across model families (Agarwal et al., 2024). We adopt the same design principle of schema-grounded workload construction, and we report strategy-level coverage and failure modes to make benchmark generation auditable beyond aggregate success rates.

Comparison with existing approaches. Table 1 contrasts PIPE-RDF with prominent KGQA benchmark generation methodologies. Unlike static public benchmarks, PIPE-RDF introduces reverse querying to ground templates in graph facts, category-aware retrieval banks to prevent structural drift, and execution-based repair loops for automated quality control. These features target schema fidelity and operational observability, two requirements that are difficult to recover from public benchmarks alone.

3 Benchmark Design Goals

PIPE-RDF is designed to meet four goals for schema-specific benchmark construction: (i) **schema fidelity** - generated queries must be grounded in a fixed schema and valid predicates; (ii) **category balance** - benchmarks should cover complex query types rather than skew toward easy factoids; (iii) **operational observability** - generation, parsing, and execution outcomes must be logged for debugging and cost control; and (iv) **maintainability** - the pipeline should be repeatable as schemas evolve, minimizing manual curation. These goals shape the pipeline phases, validation logic, and artifact outputs.

4 Category Taxonomy

To prevent evaluation bias, PIPE-RDF follows a nine-category taxonomy derived from complex

Category	SPARQL construct	Example pattern
Generic	single triple	Who is a key person at company?
Counting	COUNT	How many companies are in location?
Comparative	boolean comparison	Do company1 and company2 differ in size?
Superlative	ordering	Which company has most employees?
Ordinal	ranked	Which company was founded second earliest?
Multi-hop	ordering join chains	Which location has companies with key persons?
Intersection	conjunction	Which companies are in location and industry?
Difference	boolean exclusion	Are company1 and company2 in different industries?
Yes/No	boolean check	Is company located in location?

Table 2: Category taxonomy used for balanced benchmark generation.

KGQA datasets (Sen et al., 2022). Each category maps to distinct SPARQL constructs (e.g., aggregations, ordering, multi-hop joins) and therefore stresses different failure modes. Table 2 summarizes the taxonomy and example question patterns used in Schema C.

5 PIPE-RDF Methodology

PIPE-RDF is a three-phase pipeline (Figure 1) designed to generate schema-specific benchmarks while minimizing unanswerable or out-of-schema queries.

Phase 1: seed generation. The pipeline prompts an LLM to produce NL templates from a schema summary. Each template is validated by *reverse querying*: the system generates a SPARQL query that searches the KG for entity bindings that satisfy the template. Valid bindings instantiate the template and the resulting NL-SPARQL pairs are stored as seeds for later retrieval.

Phase 2: category-wise seeding. Templates are generated per category and paired with retrieval-augmented prompts using Phase-1 seeds. Outputs are validated by execution, optionally repaired, and stored in category-specific retrieval banks. This step ensures early coverage of complex query structures (e.g., aggregations, comparisons, multi-hop joins) and prevents domination by simple templates.

Phase 3: full dataset generation. The pipeline generates batches per category using category-specific retrieval banks. Each candidate is parsed, executed, and logged. Optional paraphrase augmentation and human validation can be enabled for robustness; the baseline run disables paraphrasing for speed. The final dataset is balanced by category

and retained in CSV/JSONL formats.

Prompt construction and retrieval. Prompts combine (i) a task description, (ii) the schema summary, (iii) a small set of retrieved NL-SPARQL examples, and (iv) the target question. Retrieval is category-aware, which narrows the space of candidate structures and reduces cross-category leakage. We keep retrieval depth small (top- $k = 2$) to control latency and prompt length.

Reverse querying and schema control. Reverse queries build deterministic binding banks for each template before generation. Each bank stores a bounded pool of valid entity/literal bindings, and Phase 3 samples from those banks client-side rather than repeatedly issuing randomized SPARQL queries. This avoids broad ORDER BY RAND() scans on large repositories and makes GraphDB work an explicit upfront stage. We also batch label and type resolution for candidate URIs with VALUES clauses, reducing thousands of small endpoint calls. We enforce explicit predicate/type whitelists and slot-type hints (company, location, person, industry) to ensure every generated query conforms to the target schema. Duplicate questions are filtered using token-level Jaccard similarity (threshold 0.99), and a stall limit prevents repeated template regeneration when no new valid rows are found.

Validation and repair. Each generated SPARQL query is parsed and executed; failures trigger a repair prompt that attempts to correct syntax and predicate usage. Repairs are logged and included in run summaries for operational tracking. The ARR-scale runs use a stricter acceptance gate than the original baseline: accepted records must parse, execute, return a non-empty answer object, and match the expected answer type for the category. Boolean ASK queries therefore produce explicit true/false answers rather than empty-result negatives. Post-generation, category-specific pattern enforcement can normalize output forms (e.g., ASK for boolean checks, DISTINCT for set-returning categories, COUNT(DISTINCT ...) for counting queries, and deterministic LIMIT 1 tie-breaking for superlative/ordinal queries). While PIPE-RDF does not require constrained decoding, it can integrate grammar-based constraints (e.g., PICARD) to improve parse validity (Scholak et al., 2021).

Human-in-the-loop controls. The pipeline supports optional manual review after each phase, with the ability to discard, edit, or inject examples. This is useful for sparse relations, sensitive attributes, or

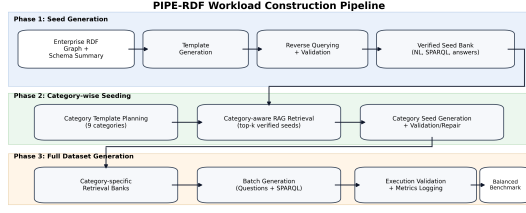


Figure 1: PIPE-RDF workload-construction pipeline: Phase 1 uses reverse query generation to build verified seeds; Phase 2 constructs category-specific seed banks; Phase 3 generates the balanced benchmark with execution-driven validation/repair.

domain-specific naming conventions where automated validation alone may not capture all semantic constraints.

6 Schemas and Implementation

Fixed schema slice. We derive a company-location mini-slice (Schema C) with 5,000 companies and three classes: `dbo:Company`, `gn:Feature` (location), and `foaf:Person` (key person), illustrated in Appendix Figure 4. The allowed predicates are `dbo:location`, `dbo:industry`, `dbo:keyPerson`, `dbo:foundingYear`, `dbo:numberOfEmployees`, and label predicates (`rdfs:label`, `spb:prefLabel`, `foaf:name`). This fixed schema avoids prompt drift and simplifies validation.

Slice extraction. The mini-slice is sampled from a public RDF source by selecting companies with at least one of the core predicates (location, industry, key person, founding year, or employee count). Linked entities (locations and key people) are included with minimal labels to support question instantiation without introducing extraneous schema elements. We intentionally exclude unrelated predicates to reduce template drift and ensure that reverse querying remains tractable.

SPB-full schema. To test the same method beyond the compact slice, we also run on a full LDBC Semantic Publishing Benchmark graph with roughly 25M triples. The graph contains many more entity types and predicates than Schema C; its top predicates include `rdf:type`, `spb:prefLabel`, `foaf:name`, `dbo:birthPlace`, `dbo:location`, `dbo:foundingYear`, `dbo:industry`, `dbo:keyPerson`, and `dbo:numberOfEmployees`. For comparability, the benchmark-generation task still targets the company/location/person-style query categories, while the schema summary and

Class	Key predicates/attributes
<code>dbo:Company</code>	<code>dbo:location</code> , <code>dbo:industry</code> , <code>dbo:keyPerson</code> , <code>dbo:foundingYear</code> , <code>dbo:numberOfEmployees</code> , <code>rdfs:label</code>
<code>gn:Feature</code>	<code>rdfs:label</code> , <code>spb:prefLabel</code>
<code>foaf:Person</code>	<code>foaf:name</code> , <code>rdfs:label</code>

Table 3: Schema C mini-slice classes and allowed predicates.

Parameter	Value
Phase 1 templates / category	5
Phase 1 seeds / template	8
Phase 2 seeds / category	20
Phase 3 targets / category	50 baseline / 200 ARR
Binding-bank pool	bounded deterministic pool
Retrieval top- <i>k</i>	2
SPARQL result cap	5
Dedup Jaccard threshold	0.99

Table 4: Key pipeline configuration. The ARR-scale runs keep the same category taxonomy while scaling Phase 3 targets and moving reverse-query work into deterministic binding-bank construction.

whitelist are derived from the larger graph. This separates graph scale and schema-profile breadth from changes in task taxonomy.

Pipeline configuration. The original baseline used 5 templates per category with 8 seed instantiations per template in Phase 1, 20 seeds per category in Phase 2, and 50 targets per category in Phase 3. The ARR-scale runs increase Phase 3 to 200 targets per category on each of two schemas (3,600 Phase-3 pairs total). Generated SPARQL is capped at 5 results to limit execution cost. Binding banks use bounded deterministic pools and client-side shuffling for diversity.

Runtime environment. We run GraphDB 10.6.3 and execute SPARQL with inference disabled. The ARR-scale generation uses Qwen 3.5 models served through vLLM and local BAAI/bge-m3 embeddings. The primary runs use Qwen/Qwen3.5-4B; robustness probes use Qwen/Qwen3.5-2B, Qwen/Qwen3.5-4B, and Qwen/Qwen3.5-9B. All runs are logged with execution latency, parse validity, and answer counts; artifacts are exported as JSONL/CSV and summarized in run-level reports. A compact reproducibility checklist is provided in Appendix Section A.1.

SPARQL formalism and schema constraints. To ensure semantic correctness, we enforce canonical query patterns:

- **Counting:** Use `COUNT(DISTINCT ?var)` to

avoid overcounting due to multiple bindings.

- **Superlative/Ordinal:** Include deterministic tie-breaking via `ORDER BY ... ?entity` to ensure reproducible results.
- **Numeric comparisons:** Cast literals to `xsd:integer` where applicable (e.g., `dbo:numberOfEmployees`).
- **Yes/No:** Use `ASK {}` for explicit boolean returns, avoiding empty-result ambiguity.

Schema constraints are enforced via predicate whitelists: `dbo:location` ranges over `gn:Feature`, `dbo:keyPerson` ranges over `foaf:Person`, and `dbo:foundingYear/dbo:numberOfEmployees` are typed literals. Label resolution prioritizes `foaf:name` for persons, `rdfs:label` for companies, and `spb:prefLabel` for locations.

7 Evaluation Protocol

PIPE-RDF logs operational and structural metrics for every generated pair. We track: (i) parse validity and execution success, (ii) non-empty answer coverage, (iii) repair attempts, (iv) answer counts, (v) prompt and question lengths, (vi) LLM latency, and (vii) SPARQL structural complexity (triple pattern counts, `FILTER` clauses, aggregations). These metrics support both data quality analysis and model evaluation. While PIPE-RDF focuses on execution-driven correctness, the artifacts can be augmented with SPARQL-specific metrics (SP-F1) and structural metrics (CodeBLEU) for model evaluation (Rony et al., 2022; Ren et al., 2020). We also record retrieval scores to analyze the relationship between example similarity and downstream execution success.

Following recent workload-generation evaluation practice (Agarwal et al., 2024), we additionally report strategy coverage and strategy-level error breakdowns to make pipeline behavior auditable beyond aggregate success rates. Strategy tags are inferred from query operators (`JOIN`, `FILTER`, `COUNT`, `ORDER`, `NEGATION`, `ASK`, `RAG-context`). `ASK` and `NEGATION` tags are retained in coverage/error matrices even when observed frequency is zero in Phase 3 outputs.

Semantic correctness audit. To verify that generated NL-SPARQL pairs are semantically aligned (not just syntactically valid), we construct a 216-pair stratified audit packet: 2 schemas $\times$ 9 categories $\times$ 12 pairs. Two annotators independently

Schema	Graph scale	Pairs	Min/Max cat.	Parse fail	Exec fail	Empty	Repairs
Schema C	38K triples, 3 types	1,800	200/200	0	0	0	0
SPB-full	25M triples, broad type/predicate profile	1,800	200/200	0	0	0	0

Table 5: Primary ARR-scale Phase-3 results. Both schemas are balanced at 200 examples per category across nine categories and pass strict parse, execution, answer-type, and category-form checks.

Schema	LLM ms	SPARQL ms	Question ms	Predicates used	Zero-count	ASK type err.
Schema C	650.5	3.7	0.05	6	0	0
SPB-full	1596.0	5.6	0.06	8	0	0

Table 6: Runtime and strict answer-shape diagnostics for the two primary runs. LLM time dominates end-to-end generation; SPARQL execution remains in the single-digit millisecond range after binding-bank construction and batched metadata lookup.

check whether (i) question intent matches the SPARQL semantics, (ii) entity bindings in the question correspond to `VALUES` clauses or equivalent query constraints, (iii) the answer type matches the expected return, and (iv) the category label matches the dominant SPARQL construct. The accompanying agreement script reports pass rate, Cohen’s κ , disagreement counts, and adjudicated error categories once annotation files are completed.

8 Benchmark Results

The primary ARR-scale evaluation uses Qwen/Qwen3.5-4B served through vLLM. We generate 1,800 Phase-3 pairs for each schema, with exactly 200 pairs in each of the nine categories. Table 5 summarizes the strict final-artifact checks. A record is accepted only if its SPARQL parses, executes, returns a category-appropriate answer object, avoids zero-count counting artifacts, and passes category-form checks such as `COUNT` for counting and ranked `ORDER BY/OFFSET` forms for ordinal queries. Both schemas satisfy these checks without repair calls.

9 Cross-Model Robustness

To test whether the pipeline depends on a single LLM size, we rerun 50-pair-per-category probes with Qwen3.5-2B, Qwen3.5-4B, and Qwen3.5-9B on both schemas. These probes use the same strict acceptance checks as the primary runs. Table 7 shows that all six probes are balanced and produce 100% parse and execution validity with no empty answers and no repairs. This does not imply that every intermediate LLM proposal is correct; rather, it shows that reverse-query grounding plus strict validation produces reliable accepted artifacts across

Schema	Model	Pairs	Parse fail	Exec fail	Empty	SPARQL ms
Schema C	Qwen3.5-2B	450	0	0	0	4.1
Schema C	Qwen3.5-4B	450	0	0	0	16.7
Schema C	Qwen3.5-9B	450	0	0	0	5.0
SPB-full	Qwen3.5-2B	450	0	0	0	8.2
SPB-full	Qwen3.5-4B	450	0	0	0	5.8
SPB-full	Qwen3.5-9B	450	0	0	0	7.2

Table 7: Cross-model generation robustness probes at 50 pairs per category per schema.

Schema	Packet size	Categories	Per cat.	Status
Schema C	108	9	12	prepared for annotation
SPB-full	108	9	12	prepared for annotation

Table 8: Semantic audit packet prepared for independent two-annotator review. Quantitative pass rate, agreement, κ , and adjudicated errors require completed annotation files.

smaller and larger open models.

10 Semantic Audit and Benchmark Utility

The semantic audit addresses the gap between executable queries and semantically correct NL–SPARQL pairs. We prepared a 216-pair audit packet sampled uniformly by schema and category: 2 schemas $\times$ 9 categories $\times$ 12 pairs. The packet includes the question, SPARQL, normalized answer, category, schema, and annotation fields for intent/query match, entity-binding correctness, answer-type correctness, category correctness, pass/fail, and error type. We also provide an agreement script that computes exact agreement and Cohen’s κ over the shared pair identifiers.

We also evaluate whether the generated benchmark is useful for NL-to-SPARQL prompting. On 180 held-out pairs per condition, we compare a schema-only zero-shot prompt against category-retrieval prompts using $k = 3$ generated examples. Table 9 shows large and consistent improvements from category retrieval. For Qwen3.5-2B, category-RAG raises answer F1 from 1.7 to 86.1 on Schema C and from 5.6 to 89.2 on SPB-full. For Qwen3.5-4B, category-RAG raises answer F1 from 32.8 to 91.1 on Schema C and from 18.3 to 90.0 on SPB-full. These results support the benchmark’s practical utility as a source of schema- and category-matched examples, not only as a static evaluation set.

11 Discussion and Operational Lessons

The reviewer concerns motivating this revision were scale, generality beyond one compact schema, LLM dependence, and semantic validation. The

Schema	Model	Setting	Parse%	Exec%	Answer F1	SP-F1
Schema C	Qwen3.5-2B	Zero-shot	45.0	45.0	1.7	82.5
Schema C	Qwen3.5-2B	Category-RAG	100.0	100.0	86.1	99.7
Schema C	Qwen3.5-4B	Zero-shot	75.0	73.3	32.8	89.9
Schema C	Qwen3.5-4B	Category-RAG	99.4	99.4	91.1	99.7
SPB-full	Qwen3.5-2B	Zero-shot	38.3	35.6	5.6	72.0
SPB-full	Qwen3.5-2B	Category-RAG	100.0	100.0	89.2	99.8
SPB-full	Qwen3.5-4B	Zero-shot	84.4	84.4	18.3	83.2
SPB-full	Qwen3.5-4B	Category-RAG	100.0	100.0	90.0	99.7

Table 9: Downstream utility evaluation on 180 held-out examples per schema/model/setting. Category-RAG uses $k = 3$ generated examples from the same schema and target category.

new results directly address the first three: the primary benchmark grows from 450 to 3,600 Phase-3 pairs; the second schema uses a much larger public RDF graph; and cross-model probes show that strict accepted artifacts are not tied to one Qwen 3.5 model size. The downstream utility experiment adds a separate usefulness claim: generated category-matched examples materially improve held-out NL-to-SPARQL prompting, especially for smaller models.

The most important operational lesson is that SPARQL endpoint design matters as much as model choice when scaling benchmark generation. The initial large-graph runs exposed endpoint bottlenecks from randomized reverse queries and many tiny metadata lookups. Replacing ORDER BY RAND() with deterministic binding banks, shuffling locally, and batching label/type retrieval reduced GraphDB load enough to complete balanced 200-per-category runs. This supports the pipeline’s methodological claim: execution grounding is practical when reverse-query work is treated as a planned data-building stage rather than an on-line sampling side effect.

The remaining semantic-audit step is intentionally separated from automated validity. Parse and execution validity prove that queries are legal and answerable; they do not prove that the natural-language question perfectly matches the SPARQL intent. The audit packet and agreement tooling make this check reproducible, but the final pass rate and κ should be reported only after independent annotations are completed.

12 Benchmark Usage

PIPE-RDF produces benchmark artifacts that can be used to evaluate NL-to-SPARQL systems on a target RDF schema. The generated pairs support execution accuracy, answer F1, SP-F1, and structural similarity evaluation. The same seed banks can also be reused as few-shot retrieval ex-

amples or synthetic supervision for schema-specific fine-tuning. The method is especially useful when public KGQA datasets do not match the target ontology, but the contribution does not depend on demonstrating adoption in any particular deployment context.

13 Conclusion

PIPE-RDF provides a practical pipeline for generating balanced, schema-specific NL-SPARQL benchmarks. Across two RDF schemas, the primary Qwen3.5-4B runs produce 3,600 strictly valid Phase-3 pairs with exact category balance, no parse failures, no execution failures, no empty answers, and no repairs. Cross-model probes with Qwen3.5-2B, 4B, and 9B show the same strict accepted-artifact profile at smaller scale, and the downstream utility evaluation shows that generated category-matched examples substantially improve NL-to-SPARQL prompting. By combining reverse querying, category-aware retrieval, binding banks, and execution-based validation, PIPE-RDF supports reproducible benchmark construction for RDF question answering.

14 Limitations

Both evaluated schemas are public. Schema C is a controlled public company-location slice and SPB-full is a public benchmark graph; we do not claim evidence from private or proprietary deployments. This is a deliberate change from an industry-track framing: the paper’s claim is about schema-specific benchmark construction, not about the prevalence or adoption of RDF/SPARQL in any particular sector.

The semantic audit is prepared but requires independent human annotation before the paper should report pass rates and Cohen’s κ . Automated parse, execution, and answer-shape checks are necessary quality gates, but they are not substitutes for semantic alignment judgments.

The downstream utility evaluation is prompt-based rather than a full comparison of fine-tuned semantic parsers. It shows that generated examples improve retrieval-augmented prompting, but future work should evaluate fine-tuning, grammar-constrained decoding, and additional RDF schemas with different ontology styles. The Qwen3.5-9B probe required a shorter 8K context on one GPU because the 16K configuration exceeded available memory; all probe prompts fit within that context,

but larger-context deployments should report serving constraints explicitly.

15 Ethical Considerations

PIPE-RDF is intended for benchmark construction over RDF graphs. When used with private graphs, deployments should follow data governance policies, minimize exposure of sensitive identifiers in prompts, and audit logs for leakage. The public schemas in this study are derived from public RDF data, and usage follows their licensing terms.

References

- Anurag Agarwal, Shiva Karthik, Chang Li, Xin Chen, Roger Casanas, Konstantinos Karanasos, Tim Kraska, Aditya Nori, Jayanta Rahman, Ce Zhang, Sparsh Mittal, Sumit Basu, and Haoran Cai. 2024. [Generating cross-model analytics workloads using llms](#). In *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, pages 1024–1034. Association for Computing Machinery.
- Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, and 12 others. 2020. [Language models are few-shot learners](#). *Preprint*, arXiv:2005.14165.
- Douwe Kiela, Max Bartolo, Yixin Nie, Divyansh Kaushik, Atticus Geiger, Zhengxuan Wu, Bertie Vidgen, Grusha Prasad, Amanpreet Singh, Pratik Ringshia, Zhiyi Ma, Tristan Thrush, Sebastian Riedel, Zeerak Waseem, Pontus Stenetorp, Robin Jia, Mohit Bansal, Christopher Potts, and Adina Williams. 2021. [Dynabench: Rethinking benchmarking in nlp](#). *Preprint*, arXiv:2104.14337.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen tau Yih, Tim Rocktaschel, Sebastian Riedel, and Douwe Kiela. 2021. [Retrieval-augmented generation for knowledge-intensive nlp tasks](#). *Preprint*, arXiv:2005.11401.
- Aleksandr Perevalov, Dennis Diefenbach, Ricardo Usbeck, and Andreas Both. 2022. [Qald-9-plus: A multilingual dataset for question answering over dbpedia and wikidata translated by native speakers](#). *Preprint*, arXiv:2202.00120.
- Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. [Codebleu: a method for automatic evaluation of code synthesis](#). *Preprint*, arXiv:2009.10297.

Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teucher, Liubov Kovriguina, and Jens Lehmann. 2022. *Sgpt: A generative approach for sparql query generation from natural language questions*. *IEEE Access*, 10:70712–70723.

Torsten Scholak, Nathan Schucher, and Dzmitry Bahdanau. 2021. *Picard: Parsing incrementally for constrained auto-regressive decoding from language models*. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 9895–9901. Association for Computational Linguistics.

Priyanka Sen, Alham Fikri Aji, and Amir Saffari. 2022. *Mintaka: A complex, natural, and multilingual dataset for end-to-end question answering*. *Preprint*, arXiv:2210.01613.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. 2023. *Llama: Open and efficient foundation language models*. *Preprint*, arXiv:2302.13971.

Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. 2017. *LC-QuAD: A Corpus for Complex Question Answering over Knowledge Graphs*, pages 210–218. Springer International Publishing.

Appendix

A Extended Operational Analysis

This appendix reports detailed diagnostics that were omitted from the main paper for ACL page-budget constraints. The key interpretation in all plots and tables is unchanged from the main text: pipeline reliability is high, and most residual difficulty is due to sparse conjunctive semantics rather than parser instability.

A.1 Reproducibility Checklist

- **Software/models:** GraphDB 10.6.3 (Docker), vLLM service, Qwen 3.5 chat models, embedding model BAAI/bge-m3.
- **Endpoint/repository:** repository `spb_company_mini_slice` with inference disabled (`infer=false`); see run log for full endpoint string.
- **Configuration:** main run uses the full-run YAML config; key sizes are listed in Table 4.
- **Randomness control:** seed control is exposed via `SEED=42` in `pipekg/config.py`. LLM generation remains stochastic; we therefore

release full run artifacts for exact reuse of produced pairs.

- **Command template:** `python scripts/run_pipeline_ollama.py` with ARR configs under `configs/arr_*.yaml`.
- **Artifacts:** `run.log`, `pipeline_records.jsonl`, phase-wise JSONL/CSV, analysis summaries, and figures under `artifacts/runs/<run_id>/`.
- **Hardware/deployment:** GraphDB in Docker plus vLLM model serving on GPU resources; per-query latencies are reported in the main paper.

Category and strategy diagnostics. Figure 2 highlights structural differences across categories, and Appendix Figure 3 reports strategy-conditioned error rates. These legacy diagnostics come from the earlier 450-pair baseline artifact; the ARR-scale strict artifacts supersede their validity and empty-answer statistics, but the plots remain useful for illustrating the diagnostic reports produced by the pipeline.

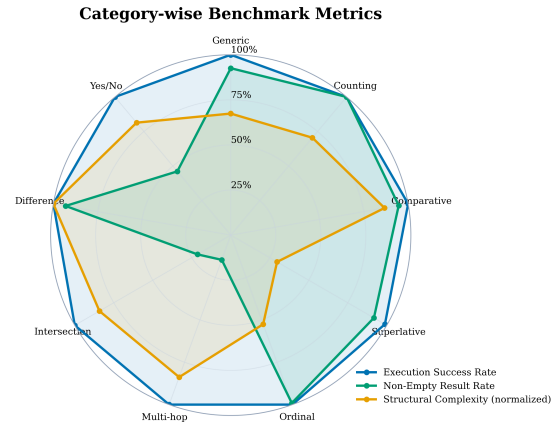


Figure 2: Category-wise metrics: execution success (100%), non-empty rate, and normalized structural complexity.

Data quality controls and throughput. In error analysis, we observed a small number of self-comparison pairs, retrieval self-reference cases, and answer-type mismatches; we implemented automated guards for each. Runtime remains generation-bound: average LLM latency is 5.3s per query versus about 7ms SPARQL latency. These measurements support production prioritization toward prompt and inference optimization rather than query engine tuning.

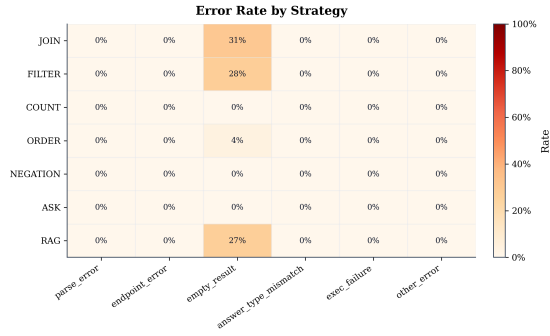


Figure 3: Phase 3 strategy-conditioned error rates. Non-zero cells are concentrated in empty-result outcomes for sparse conjunctive structures.

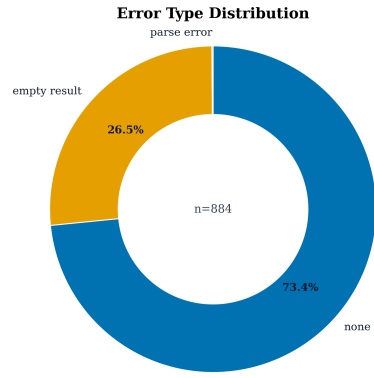


Figure 6: Error type distribution across phases.

B Additional Figures

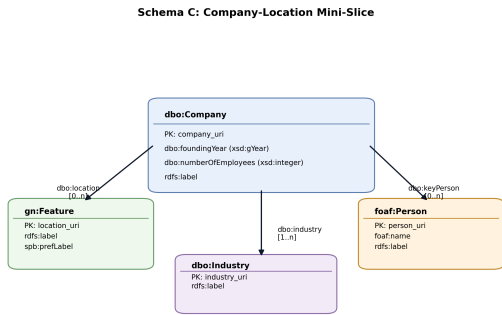


Figure 4: Schema C ontology used in this study. `dbo:Company` connects to `gn:Feature` (location), `foaf:Person` (key person), and `dbo:Industry` with typed predicates and cardinalities.

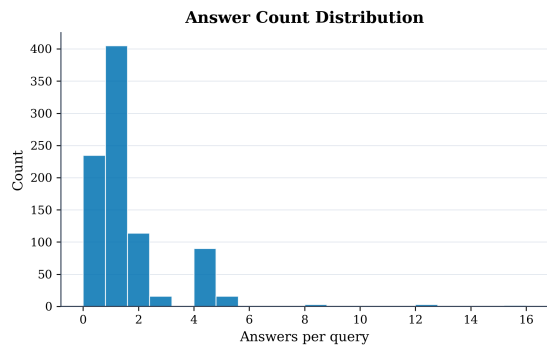


Figure 5: Answer count distribution for Phase 3.

C Example NL-SPARQL Pairs

We include one example per category from the Phase 3 benchmark. Prefix declarations are omitted for brevity.

Generic. Q: Who is a key person at (Company: Volkswagen)?

```

SELECT ?name
WHERE {
  VALUES ?company {
    ↪ <http://dbpedia.org/resource/Volkswagen> }
  ?company rdf:type dbo:Company .
  ?company dbo:keyPerson ?person .
  ?person rdf:type foaf:Person .
  ?person foaf:name ?name .
}
LIMIT 5
  
```

Counting. Q: How many companies are located in (Location: Menlo Park, California)?

```

SELECT (COUNT(DISTINCT ?company) AS ?count)
WHERE {
  VALUES ?location { <http://dbpedia.org/resourc
    ↪ ce/Menlo_Park,_California> }
  ?company a dbo:Company .
  ?company dbo:location ?location .
  ?location a gn:Feature .
}
  
```

Comparative. Q: Do (Company: Airbus) and (Company: FileMaker Inc.) have different numbers of employees?

```

ASK {
  VALUES ?company1 {
    ↪ <http://dbpedia.org/resource/Airbus> }
  VALUES ?company2 { <http://dbpedia.org/resourc
    ↪ ce/FileMaker_Inc.> }
  ?company1 a dbo:Company .
  ?company2 a dbo:Company .
}
  
```

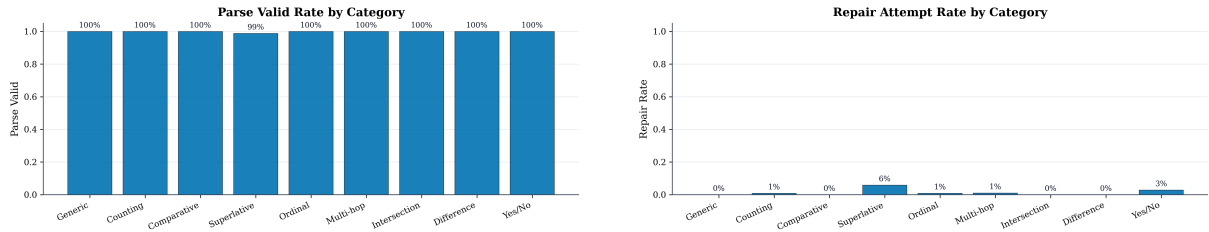


Figure 8: (Left) Parse validity by category. (Right) Repair rate by category.

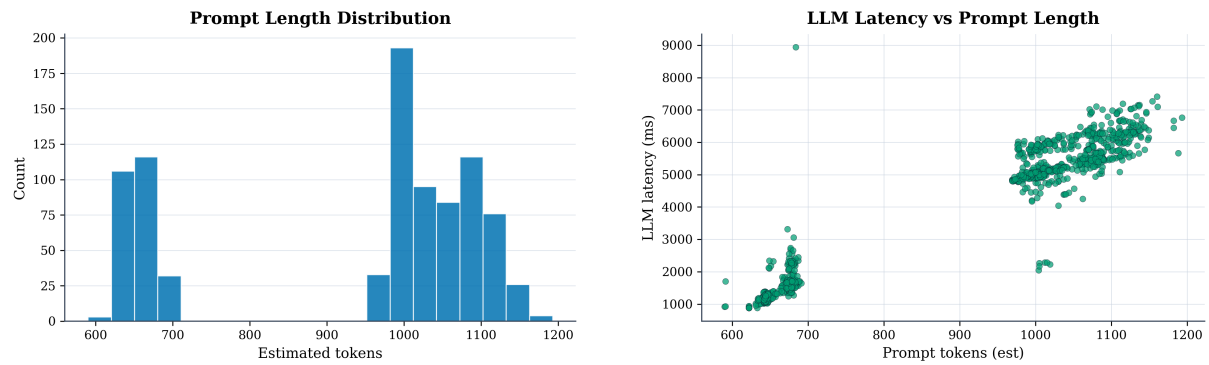


Figure 9: Prompt length distribution and latency vs. prompt length.

```
?company1 dbo:numberOfEmployees ?n1 .
?company2 dbo:numberOfEmployees ?n2 .
FILTER (xsd:integer(?n1) != xsd:integer(?n2))
}
```

Superlative. Q: Which company has the most employees?

```
SELECT ?company ?employees
WHERE {
  ?company a dbo:Company .
  ?company dbo:numberOfEmployees ?employees .
}
ORDER BY DESC(xsd:integer(?employees)) ?company
LIMIT 1
```

Ordinal. Q: Which company was founded second earliest?

```
SELECT ?company ?year
WHERE {
  ?company a dbo:Company .
  ?company dbo:foundingYear ?year .
}
ORDER BY ASC(xsd:integer(?year)) ?company
OFFSET 1
LIMIT 1
```

Multi-hop. Q: Which location contains companies that have a key person?

```
SELECT DISTINCT ?location
WHERE {
  ?company a dbo:Company .
  ?company dbo:keyPerson ?person .
  ?person rdf:type foaf:Person .
  ?company dbo:location ?location .
  ?location a gn:Feature .
}
LIMIT 5
```

Intersection. Q: Which companies are located in (Location: California) and are in the (Industry: Software)?

```
SELECT DISTINCT ?company
WHERE {
  ?company a dbo:Company .
  ?company dbo:location
    ↪ <http://dbpedia.org/resource/California> .
  ?company dbo:industry
    ↪ <http://dbpedia.org/resource/Software> .
}
LIMIT 5
```

Difference. Q: Are (Company: Airbus) and (Company: FileMaker Inc.) in different industries?

```
ASK {
  VALUES ?company1 {
    ↪ <http://dbpedia.org/resource/Airbus> }
  VALUES ?company2 { <http://dbpedia.org/resour
    ↪ ce/FileMaker_Inc.> }
  ?company1 a dbo:Company .
  ?company2 a dbo:Company .
  ?company1 dbo:industry ?industry1 .
  ?company2 dbo:industry ?industry2 .
  FILTER (?industry1 != ?industry2)
}
```

Yes/No. Q: Is (Company: Facebook) located in (Location: California)?

```
ASK {
  <http://dbpedia.org/resource/Facebook> a
    ↪ dbo:Company ;
    ↪ dbo:location
    ↪ <http://dbpedia.org/resource/California>
    ↪ .
}
```